

equity_derivatives_pricer

Pricer d'options sur actions en Python · Black-Scholes, grecques, volatilité implicite, skew · simulateur de market making

CE QUE FAIT LE PROJET

- Récupère** les données d'options d'une action (SPY, AAPL...) : prix spot, échéances, tous les calls et puts cotés — via Yahoo Finance.
- Nettoie** calcule le prix mid (bid+ask)/2, écarte les options illiquides, convertit l'échéance en maturité en années.
- Valorise** prix théorique Black-Scholes + les 5 grecques (delta, gamma, vega, theta, rho).
- Inverse le modèle** retrouve la **volatilité implicite** de chaque option à partir de son prix de marché (algorithme de Brent).
- Trace le skew** la courbe volatilité implicite = $f(\text{strike})$, qui montre que le marché ne croit pas à une volatilité constante.
- Simule un market maker** cote une fourchette bid/ask, décale son prix selon son stock (inventory skewing), exécute des trades, se couvre en delta.

LE PIPELINE EN UN COUP D'ŒIL



Chaque couche ignore les autres : le moteur de pricing ne dépend que de NumPy/SciPy (aucun réseau, aucun pandas), et le market maker reçoit un prix déjà calculé. On peut donc changer de source de données ou de modèle sans rien casser.

LES DOSSIERS

Dossier / fichier	Rôle	À quoi ça sert
market_data/	COUCHE DONNÉES	Aller chercher les données de marché et les rendre exploitables.
pricing_engine/	COUCHE MODÈLE	Les mathématiques financières pures. Le cœur du projet.
market_making/	COUCHE TRADING	La logique métier d'un desk : coter, trader, se couvrir.
visualisation/	COUCHE AFFICHAGE	Placeholder (vide) — les graphes vivent encore dans main.py.
tests/	TESTS	Validation du pricer + diagnostic de connexion Yahoo.
main.py	ORCHESTRATEUR	Le point d'entrée : câble toutes les couches sur un scénario complet.

LES CONCEPTS FINANCIERS, EN UNE LIGNE CHACUN

Concept	Traduction en français courant
Option européenne	Le droit d'acheter (call) ou de vendre (put) une action à un prix fixé, à une date fixée.
Black-Scholes	La formule qui donne le prix juste d'une option, en supposant qu'on peut la couvrir en continu.
Delta	De combien le prix de l'option bouge si l'action monte de 1 \$. C'est la grecque de la couverture.
Gamma	À quelle vitesse le delta lui-même change. Gamma élevé = il faut se recouvrir souvent.
Vega	La sensibilité à la volatilité. C'est ce qu'un desk d'options achète et vend vraiment.
Theta	L'érosion du temps : ce que l'option perd juste en vieillissant.
Volatilité implicite	La volatilité qu'il faut injecter dans Black-Scholes pour retomber sur le prix coté.
Skew	L'IV n'est pas la même à tous les strikes : elle baisse quand le strike monte. Le modèle est donc une simplification.
Delta hedging	Vendre des actions pour annuler l'exposition au sens du marché, et ne garder que la volatilité.
Inventory skewing	Baisser son prix quand on a trop de stock, pour inciter le marché à nous en débarrasser.
Parité call-put	$C - P = S - K \cdot e^{-rT}$: une égalité qui doit tenir sinon il y a arbitrage.

LES FICHIERS DE CODE, UN PAR UN

Dossier	Fichier	L.	Ce qu'il contient
pricing_engine/	black_scholes.py	79	Le cœur. Classe <code>EuropeanOption</code> : prix call/put, les 5 grecques, et <code>implied_volatility()</code> qui inverse le modèle par l'algorithme de Brent. Ne dépend que de <code>numpy</code> + <code>scipy</code> .
market_data/	data_fetcher.py	48	Classe <code>MarketDataFetcher</code> : la seule qui touche au réseau. Récupère le spot, les échéances et la chaîne d'options sur Yahoo Finance (<code>yfinance</code>).
market_data/	data_cleaner.py	47	Classe <code>MarketDataCleaner</code> : mid-price, filtre de liquidité (<code>volume > 0</code> et <code>bid > 0</code>), maturité <code>T</code> en années (<code>jours / 365.25</code>).
market_data/	data_feed.py	23	Jeu de données simulé réaliste (13 strikes autour de 540 \$) pour développer hors-ligne, sans dépendre de l'API. <i>Pas encore branché dans <code>main.py</code>.</i>
market_data/	data_enricher.py	21	Ajoute la colonne IV à toute la chaîne d'options. C'est le pont entre la couche données et le moteur de pricing. <i>Pas encore branché dans <code>main.py</code>.</i>
market_making/	pricer_mm.py	96	Classe <code>MarketMaker</code> : <code>quote_price()</code> (bid/ask décalé selon le stock), <code>execute_trade()</code> (cash + inventaire), <code>hedge_delta()</code> (couverture en actions). Gère plusieurs instruments via dictionnaires.
racine	main.py	77	Enchaîne tout : données mock → nettoyage → IV sur toute la chaîne → graphe du skew → cotation + trade + hedge sur l'option à la monnaie.
tests/	test_black_scholes.py	22	Valide le pricer par la parité call-put ($C - P = S - K \cdot e^{-rT}$), une relation d'arbitrage qui ne dépend d'aucun modèle. Passe.
tests/	test_fetcher.py	15	Script de diagnostic : vérifie que la connexion Yahoo Finance répond. Pas un test <code>pytest</code> au sens strict.
visualisation/	plots.py	0	Vide — prévu pour extraire la logique <code>matplotlib</code> de <code>main.py</code> .

OÙ EN EST LE PROJET

OK	Le pricing, les grecques et le calcul d'IV fonctionnent. Le skew calculé est réaliste (30 % à $K=480$ → 16 % à $K=590$). Le test <code>pytest</code> passe.
KO	<code>main.py</code> s'arrête avant la partie market making : <code>MarketMaker</code> a été refactorisé (ajout de <code>option_id / ticker</code>) mais les appels n'ont pas suivi.
À FAIRE	Brancher <code>data_feed.py</code> et <code>data_enricher.py</code> (aujourd'hui dupliqués dans <code>main.py</code>), remplir <code>plots.py</code> , écrire un README.

STACK TECHNIQUE

<code>numpy</code> / <code>scipy</code>	Calcul et optimisation (<code>brentq</code>)
<code>pandas</code>	Manipulation des chaînes d'options
<code>matplotlib</code>	Tracé du skew
<code>yfinance</code>	Données de marché
<code>pytest</code>	Tests automatisés